

```

using Windows.ApplicationModel.Background;
using Windows.Foundation.Collections;

namespace MyAppService
{
    public sealed class Inventory : IBackgroundTask
    {
        private BackgroundTaskDeferral backgroundTaskDeferral;
        private AppServiceConnection appServiceConnection;
        private String[] inventoryItems = new string[] { "Robot vacuum",
"Chair" };
        private double[] inventoryPrices = new double[] { 129.99, 88.99 };

        public void Run(IBackgroundTaskInstance taskInstance)
        {
            // Get a deferral so that the service isn't terminated.
            this.backgroundTaskDeferral = taskInstance.GetDeferral();

            // Associate a cancellation handler with the background task.
            taskInstance.Canceled += OnTaskCanceled;

            // Retrieve the app service connection and set up a listener for
            incoming app service requests.
            var details = taskInstance.TriggerDetails as
AppServiceTriggerDetails;
            appServiceConnection = details.AppServiceConnection;
            appServiceConnection.RequestReceived += OnRequestReceived;
        }

        private async void OnRequestReceived(AppServiceConnection sender,
AppServiceRequestReceivedEventArgs args)
        {
            // Get a deferral because we use an awaitable API below to
            respond to the message
            // and we don't want this call to get canceled while we are
            waiting.
            var messageDeferral = args.GetDeferral();

            ValueSet message = args.Request.Message;
            ValueSet returnData = new ValueSet();

            string command = message["Command"] as string;
            int? inventoryIndex = message["ID"] as int?;

            if (inventoryIndex.HasValue &&
                inventoryIndex.Value >= 0 &&
                inventoryIndex.Value < inventoryItems.GetLength(0))
            {
                switch (command)
                {
                    case "Price":
                    {
                        returnData.Add("Result",
inventoryPrices[inventoryIndex.Value]);
                        returnData.Add("Status", "OK");
                    }
                }
            }
        }
    }
}

```

```

        break;
    }

    case "Item":
    {
        returnData.Add("Result",
inventoryItems[inventoryIndex.Value]);
        returnData.Add("Status", "OK");
        break;
    }

    default:
    {
        returnData.Add("Status", "Fail: unknown
command");

        break;
    }
}
else
{
    returnData.Add("Status", "Fail: Index out of range");
}

// Return the data to the caller.
await args.Request.SendResponseAsync(returnData);

// Complete the deferral so that the platform knows that we're
done responding to the app service call.
// Note for error handling: this must be called even if
SendResponseAsync() throws an exception.
messageDeferral.Complete();
}

private void OnTaskCanceled(IBackgroundTaskInstance sender,
BackgroundTaskCancellationReason reason)
{
    if (this.backgroundTaskDeferral != null)
    {
        // Complete the service deferral.
        this.backgroundTaskDeferral.Complete();
    }
}
}
}

```

Related topics

- [Convert an app service to run in the same process as its host app](#)
- [Support your app with background tasks](#)

- [App service sample app \(C# and C++/WinRT\)](#) 

Convert an app service to run in the same process as its host app

Article • 10/20/2022

An [AppServiceConnection](#) enables another application to wake up your app in the background and start a direct line of communication with it.

With the introduction of in-process App Services, two running foreground applications can have a direct line of communication via an app service connection. App Services can now run in the same process as the foreground application which makes communication between apps much easier and removes the need to separate the service code into a separate project.

Turning an out-of-process model App Service into an in-process model requires two changes. The first is a manifest change.

XML

```
<Package
  ...
  <Applications>
    <Application Id=...
      ...
      EntryPoint="...">
        <Extensions>
          <uap:Extension Category="windows.appService">
            <uap:AppService Name="InProcessAppService" />
          </uap:Extension>
        </Extensions>
      ...
    </Application>
  </Applications>
```

Remove the `EntryPoint` attribute from the `<Extension>` element because now [OnBackgroundActivated\(\)](#) is the entry point that will be used when the app service is invoked.

The second change is to move the service logic from its separate background task project into methods that can be called from `OnBackgroundActivated()`.

Now your application can directly run your App Service. For example, in `App.xaml.cs`:

[!NOTE] The code below is different than the one provided for example 1 (out-of-process service). The code below is provided for illustration purposes only and should

not be used as part of example 2 (in-process service). To continue the article's transition from example 1 (out-of-process service) into example 2 (in-process service) continue to use the code provided for example 1 instead of the illustrative code below.

C#

```
using Windows.ApplicationModel.AppService;
using Windows.ApplicationModel.Background;
...

sealed partial class App : Application
{
    private AppServiceConnection _appServiceConnection;
    private BackgroundTaskDeferral _appServiceDeferral;

    ...

    protected override void OnBackgroundActivated(BackgroundActivatedEventArgs args)
    {
        base.OnBackgroundActivated(args);
        IBackgroundTaskInstance taskInstance = args.TaskInstance;
        AppServiceTriggerDetails appService = taskInstance.TriggerDetails as AppServiceTriggerDetails;
        _appServiceDeferral = taskInstance.GetDeferral();
        taskInstance.Canceled += OnAppServicesCanceled;
        _appServiceConnection = appService.AppServiceConnection;
        _appServiceConnection.RequestReceived += OnAppServiceRequestReceived;
        _appServiceConnection.ServiceClosed += AppServiceConnection_ServiceClosed;
    }

    private async void OnAppServiceRequestReceived(AppServiceConnection sender, AppServiceRequestReceivedEventArgs args)
    {
        AppServiceDeferral messageDeferral = args.GetDeferral();
        ValueSet message = args.Request.Message;
        string text = message["Request"] as string;

        if ("Value" == text)
        {
            ValueSet returnMessage = new ValueSet();
            returnMessage.Add("Response", "True");
            await args.Request.SendResponseAsync(returnMessage);
        }
        messageDeferral.Complete();
    }

    private void OnAppServicesCanceled(IBackgroundTaskInstance sender, BackgroundTaskCancellationReason reason)
    {
        _appServiceDeferral.Complete();
    }
}
```

```
private void AppServiceConnection_ServiceClosed(AppServiceConnection
sender, AppServiceClosedEventArgs args)
{
    _appServiceDeferral.Complete();
}
}
```

In the code above the `OnBackgroundActivated` method handles the app service activation. All of the events required for communication through an [AppServiceConnection](#) are registered, and the task deferral object is stored so that it can be marked as complete when the communication between the applications is done.

When the app receives a request and reads the [ValueSet](#) provided to see if the `key` and `Value` strings are present. If they are present then the app service returns a pair of `Response` and `True` string values back to the app on the other side of the **AppServiceConnection**.

Learn more about connecting and communicating with other apps at [Create and Consume an App Service](#).

Extend your app with services, extensions, and packages

Article • 10/20/2022

There are many technologies in Windows 10 for extending and componentizing your app. This table should help you determine which technology you should use depending on requirements. It is followed by a brief description of the scenarios and technologies.

Scenario	Resource package	Asset package	Optional package	Flat bundle	App Extension	App service	Streaming Install
Third-party code plug-ins					✓		
In-proc code plug-ins			✓				
UX Assets (strings/images)	✓	✓	✓		✓		✓
On demand content (for example, additional Levels)			✓		✓		✓
Separate licensing and acquisition			✓		✓	✓	
In-app acquisition			✓		✓		
Optimize install time	✓		✓		✓		✓
Reduce disk footprint	✓		✓				
Optimize packaging		✓	✓	✓			
Reduce publishing time	✓	✓	✓	✓			

Scenario descriptions (the rows in the table above)

Third-party plug-ins

Code that you can download from the store and run from your app. For example, extensions for the Microsoft Edge browser.

In-proc code plug-ins

Code that runs in-process with your app. May also include content. Because the code runs in-process, a higher level of trust is assumed. You may choose not to expose this kind of extensibility to a third-party.

UX Assets (string/images)

User interface assets such as localized strings, images, and any other UI content that you want to factor based on locale or any other reason.

On demand content

Content that you want to download at a later time. For example, in-app purchases that allow you to download new levels, skins, or functionality.

Separate licensing and acquisition

The ability to license and acquire the content independently of the app.

In-app acquisition

Indicates whether there is programmatic support to acquire the content from within the app.

Optimize install time

Provides functionality to decrease the time it takes to acquire the app from the store and start running.

Reduce disk footprint Reduces the size of an app by only including necessary apps or resources.

Optimize packaging Optimizes the app packaging process for large-scale or complex apps.

Reduce publishing time Minimize the amount of time it takes to publish your app in the Store, local share, or web server.

Technology descriptions (the columns in the table above)

Resource package

Resource packages are asset-only packages that allow your app to adapt to multiple display sizes and system languages. The resource package targets user language, system scale, and DirectX features, allowing the app to be tailored to a variety of user scenarios. Although an app package can contain several resources, the OS will only download the relevant resources per user device, saving bandwidth and disk space.

Asset package Asset packages are a common, centralized source of executable, or non-executable files for use by your app. These are typically non-processor or language-specific files. For example, this might include a collection of pictures in one asset package, and videos in another asset package, both of which are used by the app. If your app supports multiple architectures and multiple languages, these assets could be included in the architecture package or resource package, but that also means the assets would be duplicated multiple times across the various architecture packages, taking up disk space. If asset packages are used, they only need to be included in the overall app package once. See [Introduction to asset packages](#) to learn more.

Optional package

Optional packages are used to either supplement or extend the original functionality of an app package. It's possible to publish an app, followed by publishing optional packages at a later time, or to publish both the app and optional packages simultaneously. By extending your app via an optional package, you have the advantages of distributing and monetizing content as a separate app package. Optional packages are typically intended to be developed by the original app developer, since they run with the identity of the main app (unlike app extensions). Depending on how you define your optional package, you can load code, assets, or code and assets from your optional package to your main app. If you need to enhance your app with content that can be monetized, licensed, and distributed separately, then optional packages might be the right choice for you. For implementation details, see [Optional packages and related set authoring](#).

Flat bundle [Flat bundle app packages](#) are similar to regular app bundles, except that instead of including all of the app packages within the folder, the flat bundle only contains *references* to those app packages. By containing references to app packages instead of the files themselves, a flat bundle will reduce the amount of time it takes to package and download an app.

App Extension

[App extensions](#) enable your UWP app to host content provided by other UWP apps. Discover, enumerate, and access read-only content from those apps.

If an app supports extensions, any developer can submit an extension for the app. Thus, the host app needs to be robust when it loads an extension that it hasn't been pre-tested with. Extensions should be considered untrusted.

Applications cannot load code from extensions. If you need code execution, consider App Services.

App Service

Windows app services enable app-to-app communication by allowing your UWP app to provide services to another Universal Windows app. App services let you create UI-less services that apps can call on the same device, and starting with Windows 10, version 1607, on remote devices. See [Create and consume an app service](#) for details.

App services are UWP apps that provide services to other UWP apps. They are analogous to web services on a device. An app service runs as a background task in the host app and can provide its service to other apps. For example, an app service might provide a bar code scanner service that other apps could use. Or perhaps an Enterprise suite of apps has a common spell checking app service that is available to the other apps in the suite.

UWP App Streaming install

Streaming Install is a way to optimize how your app is delivered to users. Rather than waiting for the entire app to download before you can use it, users can engage with the app as soon as a required portion has been downloaded. It's up to you, as a developer, to segment your app into a required section for basic activation and launch and additional content for the rest of the app. See [UWP App Streaming Install](#) for more information and implementation details.

See Also

[Create and consume an app service](#)

[Introduction to asset packages](#)

[Package creation with the packaging layout](#)

[Optional packages and related set authoring](#)

[Developing with asset packages and package folding](#)

[UWP App Streaming Install](#)

Flat bundle app packages

Windows.ApplicationModel.AppService namespace

Windows.ApplicationModel.Extensions namespace

Create and host an app extension

Article • 10/26/2022

This article shows you how to create a Windows 10 app extension and host it in an app. App extensions are supported in UWP apps and [packaged desktop apps](#).

To demonstrate how to create an app extension, this article uses package manifest XML and code snippets from the [Math Extension code sample](#)[↗]. This sample is a UWP app, but the features demonstrated in the sample are also applicable to packaged desktop apps. Follow these instructions to get started with the sample:

- Download and unzip the [Math Extension code sample](#)[↗].
- In Visual Studio 2019, open MathExtensionSample.sln. Set the build type to x86 (**Build > Configuration Manager**, then change **Platform** to **x86** for both projects).
- Deploy the solution: **Build > Deploy Solution**.

Introduction to app extensions

In Windows 10, app extensions provide functionality similar to what plug-ins, add-ins, and add-ons do on other platforms. App extensions were introduced in the Windows 10 Anniversary edition (version 1607, build 10.0.14393).

App extensions are UWP apps or packaged desktop apps that have an extension declaration that allows them to share content and deployment events with a host app. An extension app can provide multiple extensions.

Because app extensions are just UWP apps or packaged desktop apps, they can also be fully functional apps, host extensions, and provide extensions to other apps--all without creating separate app packages.

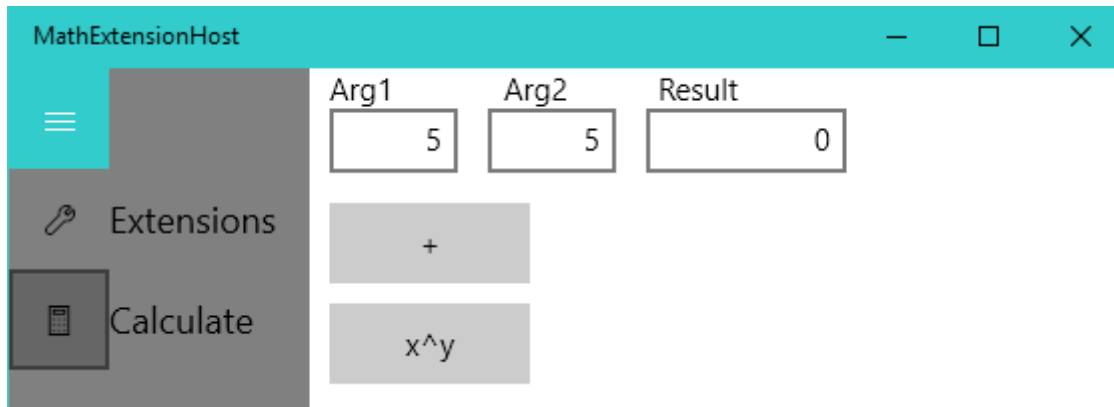
When you create an app extension host, you create an opportunity to develop an ecosystem around your app in which other developers can enhance your app in ways that you may not have expected or had the resources for. Consider Microsoft Office extensions, Visual Studio extensions, browser extensions, etc. These create richer experiences for those apps that go beyond the functionality they shipped with. Extensions can add value and longevity to your app.

At a high level, to set up an app extension relationship, we need to:

1. Declare an app to be an extension host.
2. Declare an app to be an extension.

3. Decide whether to implement the extension as an app service, background task, or some other way.
4. Define how the hosts and its extensions will communicate.
5. Use the [Windows.ApplicationModel.AppExtensions](#) API in the host app to access the extensions.

Let's see how this is done by examining the [Math Extension code sample](#) which implements a hypothetical calculator that you can add new functions to by using extensions. In Microsoft Visual Studio 2019, load **MathExtensionSample.sln** from the code sample.



Declare an app to be an extension host

An app identifies itself as an app extension host by declaring the `<AppExtensionHost>` element in its `Package.appxmanifest` file. See the `Package.appxmanifest` file in the **MathExtensionHost** project to see how this is done.

Package.appxmanifest in the MathExtensionHost project

XML

```
<Package
...
xmlns:uap3="http://schemas.microsoft.com/appx/manifest/uap/windows10/3"
IgnorableNamespaces="uap uap3 mp">
...
  <Applications>
    <Application Id="App" ... >
      ...
      <Extensions>
        <uap3:Extension Category="windows.appExtensionHost">
          <uap3:AppExtensionHost>
            <uap3:Name>com.microsoft.mathext</uap3:Name>
          </uap3:AppExtensionHost>
        </uap3:Extension>
      </Extensions>
    </Application>
```

```
    </Applications>
    ...
</Package>
```

Notice the `xmlns:uap3="http://..."` and the presence of `uap3` in `IgnorableNamespaces`. These are necessary because we are using the `uap3` namespace.

`<uap3:Extension Category="windows.appExtensionHost">` identifies this app as an extension host.

The **Name** element in `<uap3:AppExtensionHost>` is the *extension contract* name. When an extension specifies the same extension contract name, the host will be able to find it. By convention, we recommend building the extension contract name using your app or publisher name to avoid potential collisions with other extension contract names.

You can define multiple hosts and multiple extensions in the same app. In this example, we declare one host. The extension is defined in another app.

Declare an app to be an extension

An app identifies itself as an app extension by declaring the `<uap3:AppExtension>` element in its `Package.appxmanifest` file. Open the `Package.appxmanifest` file in the **MathExtension** project to see how this is done.

Package.appxmanifest in the MathExtension project:

XML

```
<Package
  ...
  xmlns:uap3="http://schemas.microsoft.com/appx/manifest/uap/windows10/3"
  IgnorableNamespaces="uap uap3 mp">
  ...
  <Applications>
    <Application Id="App" ... >
      ...
      <Extensions>
        ...
        <uap3:Extension Category="windows.appExtension">
          <uap3:AppExtension Name="com.microsoft.mathext"
            Id="power"
            DisplayName="x^y"
            Description="Exponent"
            PublicFolder="Public">
            <uap3:Properties>
              <Service>com.microsoft.powservice</Service>
            </uap3:Properties>
```

```

        </uap3:AppExtension>
    </uap3:Extension>
</Extensions>
</Application>
</Applications>
...
</Package>

```

Again, notice the `xmlns:uap3="http://..."` line, and the presence of `uap3` in `IgnorableNamespaces`. These are necessary because we are using the `uap3` namespace.

`<uap3:Extension Category="windows.appExtension">` identifies this app as an extension.

The meaning of the `<uap3:AppExtension>` attributes are as follows:

Attribute	Description	Required
Name	This is the extension contract name. When it matches the Name declared in a host, that host will be able to find this extension.	✓
ID	Uniquely identifies this extension. Because there can be multiple extensions that use the same extension contract name (imagine a paint app that supports several extensions), you can use the ID to tell them apart. App extension hosts can use the ID to infer something about the type of extension. For example, you could have one Extension designed for desktop and another for mobile, with the ID being the differentiator. You could also use the Properties element, discussed below, for that.	✓
DisplayName	Can be used from your host app to identify the extension to the user. It is queryable from, and can use, the new resource management system (<code>ms-resource:TokenName</code>) for localization. The localized content is loaded from the app extension package, not the host app.	
Description	Can be used from your host app to describe the extension to the user. It is queryable from, and can use, the new resource management system (<code>ms-resource:TokenName</code>) for localization. The localized content is loaded from the app extension package, not the host app.	
PublicFolder	The name of a folder, relative to the package root, where you can share content with the extension host. By convention the name is "Public", but you can use any name that matches a folder in your extension.	✓

`<uap3:Properties>` is an optional element that contains custom metadata that hosts can read at runtime. In the code sample, the extension is implemented as an app service so host needs a way to get the name of that app service so it can call it. The name of the

app service is defined in the <Service> element, which we defined (we could have called it anything we wanted). The host in the code sample looks for this property at runtime to learn the name of the app service.

Decide how you will implement the extension.

The Build 2016 session about app extensions demonstrates how to use the public folder that is shared between the host and the extensions. In that example, the extension is implemented by a JavaScript file that is stored in the public folder, which the host invokes. That approach has the advantage of being lightweight, does not require compilation, and can support making the default landing page that provides instructions for the extension and a link to the host app's Microsoft Store page. See the [Build 2016 app extension code sample](#) for details. Specifically, see the **InvertImageExtension** project and `InvokeLoad()` in `ExtensionManager.cs` in the **ExtensibilitySample** project.

In this example, we'll use an app service to implement the extension. App services have the following advantages:

- If the extension crashes, it won't take down the host app because the host app runs in its own process.
- You can use the language of your choice to implement the service. It doesn't have to match language used to implement the host app.
- The app service has access to its own app container--which may have different capabilities than the host has.
- There is isolation between data in service and the host app.

Host app service code

Here is the host code that invokes the extension's app service:

ExtensionManager.cs in the MathExtensionHost project

C#

```
public async Task<double> Invoke(ValueSet message)
{
    if (Loaded)
    {
        try
        {
            // make the app service call
            using (var connection = new AppServiceConnection())
            {
```



```

        // service name is defined in appxmanifest properties
        connection.AppServiceName = _serviceName;
        // package Family Name is provided by the extension
        connection.PackageFamilyName =
AppExtension.Package.Id.FamilyName;

        // open the app service connection
        AppServiceConnectionStatus status = await
connection.OpenAsync();
        if (status != AppServiceConnectionStatus.Success)
        {
            Debug.WriteLine("Failed App Service Connection");
        }
        else
        {
            // Call the app service
            AppServiceResponse response = await
connection.SendMessageAsync(message);
            if (response.Status == AppServiceResponseStatus.Success)
            {
                ValueSet answer = response.Message as ValueSet;
                if (answer.ContainsKey("Result")) // When our app
service returns "Result", it means it succeeded
                {
                    return (double)answer["Result"];
                }
            }
        }
    }
    catch (Exception)
    {
        Debug.WriteLine("Calling the App Service failed");
    }
}
return double.NaN; // indicates an error from the app service
}

```

This is typical code for invoking an app service. For details on how to implement and call an app service, see [How to create and consume an app service](#).

One thing to note is how the name of the app service to call is determined. Because the host doesn't have information about the implementation of the extension, the extension needs to provide the name of its app service. In the code sample, the extension declares the name of the app service in its file in the `<uap3:Properties>` element:

Package.appxmanifest in the MathExtension project

XML

```

...
<uap3:Extension Category="windows.appExtension">
  <uap3:AppExtension ...>
    <uap3:Properties>
      <Service>com.microsoft.powservice</Service>
    </uap3:Properties>
  </uap3:AppExtension>
</uap3:Extension>

```

You can define your own XML in the `<uap3:Properties>` element. In this case, we define the name of the app service so that the host can use it when it invokes the extension.

When the host loads an extension, code like this extracts the name of the service from the properties defined in the extension's Package.appxmanifest:

Update() in ExtensionManager.cs, in the MathExtensionHost project

C#

```

...
var properties = await ext.GetExtensionPropertiesAsync() as PropertySet;

...
#region Update Properties
// update app service information
_serviceName = null;
if (_properties != null)
{
    if (_properties.ContainsKey("Service"))
    {
        PropertySet serviceProperty = _properties["Service"] as PropertySet;
        this._serviceName = serviceProperty["#text"].ToString();
    }
}
#endregion

```

With the name of the app service stored in `_serviceName`, the host is able to use it to invoke the app service.

Calling an app service also requires the family name of the package that contains the app service. Fortunately, the app extension API provides this information which is obtained in the line: `connection.PackageFamilyName =`

```
AppExtension.Package.Id.FamilyName;
```

Define how the host and the extension will communicate

App services use a [ValueSet](#) to exchange information. As the author of the host, you need to come up with a protocol for communicating with extensions that is flexible. In the code sample, that means accounting for extensions that may take 1, 2, or more arguments in the future.

For this example, the protocol for the arguments is a **ValueSet** containing the key value pairs named 'Arg' + the argument number, for example, `Arg1` and `Arg2`. The host passes all of the arguments in the **ValueSet**, and the extension makes use of the ones that it needs. If the extension is able to calculate a result, then the host expects the **ValueSet** returned from the extension to have a key named `Result` that contains the value of the calculation. If that key is not present, the host assumes that the extension couldn't complete the calculation.

Extension app service code

In the code sample, the extension's app service is not implemented as a background task. Instead, it uses the single proc app service model in which the app service runs in the same process as the extension app that hosts it. This is still a different process from the host app, providing the benefits of process separation, while gaining some performance benefits by avoiding cross-process communication between the extension process and the background process that implements the app service. See [Convert an app service to run in the same process as its host app](#) to see the difference between an app service that runs as a background task versus in the same process.

The system makes to `OnBackgroundActivate()` when the app service is activated. That code sets up event handlers to handle the actual app service call when it comes (`OnAppServiceRequestReceived()`) as well as deal with housekeeping events such as getting a deferral object handling a cancel or closed event.

App.xaml.cs in the MathExtension project.

C#

```
protected override void OnBackgroundActivated(BackgroundActivatedEventArgs args)
{
    base.OnBackgroundActivated(args);

    if ( _appServiceInitialized == false ) // Only need to setup the handlers once
    {
        _appServiceInitialized = true;

        IBackgroundTaskInstance taskInstance = args.TaskInstance;
        taskInstance.Canceled += OnAppServicesCanceled;
```

```

        AppServiceTriggerDetails appService = taskInstance.TriggerDetails as
AppServiceTriggerDetails;
        _appServiceDeferral = taskInstance.GetDeferral();
        _appServiceConnection = appService.AppServiceConnection;
        _appServiceConnection.RequestReceived +=
OnAppServiceRequestReceived;
        _appServiceConnection.ServiceClosed +=
AppServiceConnection_ServiceClosed;
    }
}

```

The code that does the work of the extension is in `OnAppServiceRequestReceived()`. This function is called when the app service is invoked to perform a calculation. It extracts the values it needs from the **ValueSet**. If it can do the calculation, it puts the result, under a key named **Result**, in the **ValueSet** that is returned to the host. Recall that according to the protocol defined for how this host and its extensions will communicate, the presence of a **Result** key will indicate success; otherwise failure.

App.xaml.cs in the MathExtension project.

C#

```

private async void OnAppServiceRequestReceived(AppServiceConnection sender,
AppServiceRequestReceivedEventArgs args)
{
    // Get a deferral because we use an awaitable API below
    (SendResponseAsync()) to respond to the message
    // and we don't want this call to get cancelled while we are waiting.
    AppServiceDeferral messageDeferral = args.GetDeferral();
    ValueSet message = args.Request.Message;
    ValueSet returnMessage = new ValueSet();

    double? arg1 = Convert.ToDouble(message["arg1"]);
    double? arg2 = Convert.ToDouble(message["arg2"]);
    if (arg1.HasValue && arg2.HasValue)
    {
        returnMessage.Add("Result", Math.Pow(arg1.Value, arg2.Value)); //
        For this sample, the presence of a "Result" key will mean the call succeeded
    }

    await args.Request.SendResponseAsync(returnMessage);
    messageDeferral.Complete();
}

```

Manage extensions

Now that we've seen how to implement the relationship between a host and its extensions, let's see how a host finds extensions installed on the system and reacts to the addition and removal of packages containing extensions.

The Microsoft Store delivers extensions as packages. The [AppExtensionCatalog](#) finds installed packages that contain extensions matching the host's extension contract name, and supplies events that fire when an app extension package relevant to the host is installed or removed.

In the code sample, the `ExtensionManager` class (defined in `ExtensionManager.cs` in the **MathExtensionHost** project) wraps the logic for loading extensions and responding to extension package installs and uninstalls.

The `ExtensionManager` constructor uses the `AppExtensionCatalog` to find the app extensions on the system that have the same extension contract name as the host:

ExtensionManager.cs in the MathExtensionHost project.

C#

```
public ExtensionManager(string extensionContractName)
{
    // catalog & contract
    ExtensionContractName = extensionContractName;
    _catalog = AppExtensionCatalog.Open(ExtensionContractName);
    ...
}
```

When an extension package is installed, the `ExtensionManager` gathers information about the extensions in the package that have the same extension contract name as the host. An install may represent an update in which case the affected extension's information is updated. When an extension package is uninstalled, the `ExtensionManager` removes information about the affected extensions so that the user knows which extensions are no longer available.

The `Extension` class (defined in `ExtensionManager.cs` in the **MathExtensionHost** project) was created for the code sample to access an extension's ID, description, logo, and app specific information such as whether the user has enabled the extension.

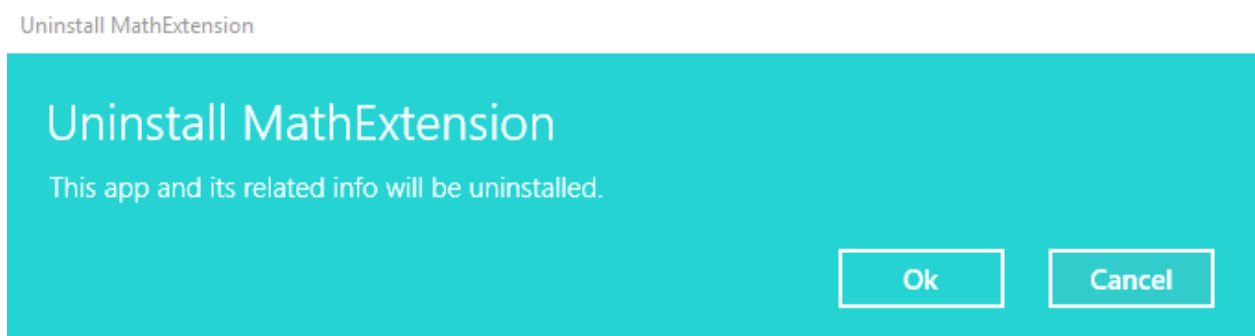
To say that the extension is loaded (see `Load()` in `ExtensionManager.cs`) means that the package status is fine and that we've obtained its ID, logo, description, and public folder (which we don't use in this sample-its just to show how you get it). The extension package itself isn't being loaded.

The concept of unloading is used for keeping track of which extensions should no longer be presented to the user.

The `ExtensionManager` provides a collection `Extension` instances so that the extensions, their names, descriptions, and logos can be data bound to UI. The **ExtensionsTab** page binds to this collection and provides UI for enabling/disabling extensions as well as removing them.



When an extension is removed, the system prompts the user to verify that they want to uninstall the package that contains the extension (and possibly contains other extensions). If the user agrees, the package is uninstalled and the `ExtensionManager` removes the extensions in the uninstalled package from the list of extensions available to the host app.



Debugging app extensions and hosts

Often, the extension host and extension are not part of the same solution. In that case, to debug the host and the extension:

1. Load your host project in one instance of Visual Studio.
2. Load your extension in another instance of Visual Studio.
3. Launch your host app in the debugger.
4. Launch the extension app in the debugger. (If you want to deploy the extension, rather than debug it, to test the host's package install event, do **Build > Deploy Solution**, instead).

Now you will be able to hit breakpoints in the host and the extension. If you start debugging the extension app itself, you will see a blank window for the app. If you don't want to see the blank window, you can change the debugging settings for the extension project to not launch the app but instead debug it when it starts (right-click the extension project, **Properties** > **Debug** > select **Do not launch, but debug my code when it starts**) You'll still need to start debugging (F5) the extension project, but it will wait until the host activates the extension and then your breakpoints in the extension will be hit.

Debug the code sample

In the code sample, the host and the extension are in the same solution. Do the following to debug:

1. Ensure that **MathExtensionHost** is the startup project (right-click the **MathExtensionHost** project, click **Set as StartUp project**).
2. Put a breakpoint on `Invoke` in `ExtensionManager.cs`, in the **MathExtensionHost** project.
3. F5 to run the **MathExtensionHost** project.
4. Put a breakpoint on `OnAppServiceRequestReceived` in `App.xaml.cs` in the **MathExtension** project.
5. Start debugging the **MathExtension** project (right-click the **MathExtension** project, **Debug** > **Start new instance**) which will deploy it and trigger the package install event in the host.
6. In the **MathExtensionHost** app, navigate to the **Calculation** page, and click **x^y** to activate the extension. The `Invoke()` breakpoint is hit first and you can see the extensions app service call being made. Then the `OnAppServiceRequestReceived()` method in the extension is hit, and you can see the app service calculate the result and return it.

Troubleshooting extensions implemented as an app service

If your extension host has trouble connecting to the app service for your extension, ensure that the `<uap:AppService Name="...">` attribute matches what you put in your `<Service>` element. If they don't match, the service name that your extension provides the host won't match the app service name you implemented, and the host won't be able to activate your extension.

Package.appxmanifest in the MathExtension project:

XML

```

<Extensions>
  <uap:Extension Category="windows.appService">
    <uap:AppService Name="com.microsoft.sqrtservice" />      <!-- This must
match the contents of <Service>...</Service> -->
  </uap:Extension>
  <uap3:Extension Category="windows.appExtension">
    <uap3:AppExtension Name="com.microsoft.mathext" Id="sqrt"
DisplayName="Sqrt(x)" Description="Square root" PublicFolder="Public">
      <uap3:Properties>
        <Service>com.microsoft.powservice</Service>    <!-- this must match
<uap:AppService Name=...> -->
      </uap3:Properties>
    </uap3:AppExtension>
  </uap3:Extension>
</Extensions>

```

A checklist of basic scenarios to test

When you build an extension host and are ready to test how well it supports extensions, here are some basic scenarios to try:

- Run the host and then deploy an extension app
 - Does the host pick up new extensions that come along while its running?
- Deploy the extension app and then deploy and run the host.
 - Does the host pick up previously-existing extensions?
- Run the host and then remove extension app.
 - Does the host detect the removal correctly?
- Run the host and then update the extension app to a newer version.
 - Does the host pick up the change and unload the old versions of the extension properly?

Advanced scenarios to test:

- Run the host, move the extension app to removable media, remove the media
 - Does the host detect the change in package status and disable the extension?
- Run the host, then corrupt the extension app (make it invalid, signed differently, etc.)
 - Does the host detect the tampered extension and handle it correctly?
- Run the host, then deploy an extension app that has invalid content or properties
 - Does the host detect invalid content and handle it correctly?

Design considerations

- Provide UI that shows the user which extensions are available and allows them to enable/disable them. You might also consider adding glyphs for extensions that become unavailable because a package goes offline, etc.
- Direct the user to where they can get extensions. Perhaps your extension page can provide a Microsoft Store search query that brings up a list of extensions that can be used with your app.
- Consider how to notify the user of the addition and removal of extensions. You might create a notification for when a new extension is installed and invite the user to enable it. Extensions should be disabled by default so that users are in control.

How app extensions differ from optional packages

The key differentiator between [optional packages](#) and app extensions are open ecosystem versus closed ecosystem, and dependent package versus independent package.

App extensions participate in an open ecosystem. If your app can host app extensions, anyone can write an extension for your host as long as they comply with your method of passing/receiving information from the extension. This differs from optional packages which participate in a closed ecosystem where the publisher decides who is allowed to make an optional package that can be used with the app.

App extensions are independent packages and can be standalone apps. They cannot have a deployment dependency on another app. Optional packages require the primary package and cannot run without it.

An expansion pack for a game would be a good candidate for an optional package because it's tightly bound to the game, it can't run independently of the game, and you may not want expansion packs to be created by just any developer in the ecosystem.

If that same game had customizable UI add-ons or theming, then an app extension could be a good choice because the app providing the extension could run on its own, and any 3rd party could make them.

Remarks

This topic provides an introduction to app extensions. The key things to note are the creation of the host and marking it as such in its Package.appxmanifest file, creating the extension and marking it as such in its Package.appxmanifest file, determining how to implement the extension (such as an app service, background task, or other means),

defining how the host will communicate with extensions, and using the [AppExtensions API](#) to access and manage extensions.

Related topics

- [Introduction to App Extensions](#)
- [Build 2016 session about app extensions](#)
- [Build 2016 app extension code sample](#) 
- [Support your app with background tasks](#)
- [How to create and consume an app service.](#)
- [AppExtensions namespace](#)
- [Extend your app with services, extensions, and packages](#)

Create and host a package extension

Article • 10/10/2023

This article shows you how to create a Windows 11 package extension and host it in an app. Package extensions are supported in UWP apps and [packaged desktop apps](#).

Package extensions are like app extension but come with more flexibility. Any package with an extension declaration can now seamlessly share content and deployment events with a host app. This capability extends beyond app packages, encompassing main, optional, framework, and resource packages. While package extensions are declared with a different set of XML elements, the implementation, management, and debugging of package extensions are the same application extensions. For more information about application extensions, see [Create and host an app extension](#).

Package extension is available on Windows 11 Server now and will be available for Windows 11 in a future release.

Declare a package to be an extension host

An app identifies itself as a package extension host by declaring the `<PkgExtensionHost>` element in its **Package.appxmanifest** file.

The following shows an example package extension declaration in a Package.appxmanifest file.

XML

```
<Package
  ...
  xmlns:uap17="http:schemas.microsoft.com/appx/manifest/uap/windows10/17"
  IgnorableNamespaces="uap uap17">
  ...
  <Extensions>
    <uap17:Extension Category="windows.packageExtensionHost">
      <uap17:PackageExtensionHost>
        <uap17:Name>com.microsoft.mathext</uap17:Name>
      </uap17:PackageExtensionHost>
    </uap17:Extension>
  </Extensions>
  ...
</Package>
```

Notice the `xmlns:uap17="http://..."` and the presence of `uap17` in `IgnorableNamespaces`. These are necessary because we are using the `uap17`

namespace.

`<uap17:Extension Category="windows.packageExtensionHost">` identifies this package as an extension host.

The `Name` element in `<uap17:PackageExtensionHost>` is the extension contract name.

When an extension specifies the same extension contract name, the host will be able to find it. By convention, we recommend building the extension contract name using your app or publisher name to avoid potential collisions with other extension contract names.

You can define multiple hosts and multiple extensions in the same package. In this example, we declare one host. The extension is defined in another package.

Declare a package to be an extension

A package identifies itself as a package extension by declaring the `<uap17:PackageExtension>` element in its `Package.appxmanifest` file. The following example shows an example package declaring itself as an extension

XML

```
<Package
...
xmlns:uap17="http:schemas.microsoft.com/appx/manifest/uap/windows10/17"
IgnorableNamespaces="uap uap17">
...
<Extensions>
  <uap17:Extension Category="windows.packageExtension">
    <uap17:PackageExtension Name="com.microsoft.ai.aiexplorer.model"
      Id="power"
      DisplayName="x^y"
      PublicFolder="Public"
      Description="Exponent">
      <uap17:Properties>
        <Service>com.microsoft.powservice</Service>
      </uap17:Properties>
    </uap17:PackageExtension>
  </uap17:Extension>
</Extensions>
...
</Package>
```

Again, notice the `xmlns:uap17="http://..."` line, and the presence of `uap17` in `IgnorableNamespaces`. These are necessary because we are using the `uap17` namespace.

`<uap17:Extension Category="windows.packageExtension">` identifies this package as an extension.

The meaning of the `<uap17:PackageExtension>` attributes are as follows

 Expand table

Attribute	Description	Required
Name	This is the extension contract name. When it matches the Name declared in a host, that host will be able to find this extension.	✓
ID	Uniquely identifies this extension. Because there can be multiple extensions that use the same extension contract name (imagine a paint app that supports several extensions), you can use the ID to tell them apart. App extension hosts can use the ID to infer something about the type of extension. For example, you could have one Extension designed for desktop and another for mobile, with the ID being the differentiator. You could also use the Properties element, discussed below, for that.	✓
DisplayName	Can be used from your host app to identify the extension to the user. It is queryable from, and can use, the new resource management system (<code>ms-resource:TokenName</code>) for localization. The localized content is loaded from the app extension package, not the host app.	
Description	Can be used from your host app to describe the extension to the user. It is queryable from, and can use, the new resource management system (<code>ms-resource:TokenName</code>) for localization. The localized content is loaded from the app extension package, not the host app.	
PublicFolder	The name of a folder, relative to the package root, where you can share content with the extension host. By convention the name is "Public", but you can use any name that matches a folder in your extension.	✓

`<uap17:Properties>` is an optional element that contains custom metadata that hosts can read at runtime. In the code sample, the extension is implemented as an app service so host needs a way to get the name of that app service so it can call it. The name of the app service is defined in the `<Service>` element, which we defined (we could have called it anything we wanted). The host in the code sample looks for this property at runtime to learn the name of the app service.

Remarks

This topic provides an introduction to package extensions. The implementation, management, and debugging of Package Extension is similar to that of App Extension and can be referred to from [here](#). The key things to note are the creation of the host and marking it as such in its Package.appxmanifest file, creating the extension and marking it as such in its Package.appxmanifest file, determining how to implement the extension (such as an app service, background task, COM or WinRT server, or other means), defining how the host will communicate with extensions, and using the PackageExtensions API to access and manage extensions.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Create hosted apps

Article • 05/26/2023

Starting in Windows 10, version 2004, you can create *hosted apps*. A hosted app shares the same executable and definition as a parent *host* app, but it looks and behaves like a separate app on the system.

Hosted apps are useful for scenarios where you want a component (such as an executable file or a script file) to behave like a standalone Windows 10 app, but the component requires a host process in order to execute. For example, a PowerShell or Python script could be delivered as a hosted app that requires a host to be installed in order to run. A hosted app can have its own start tile, identity, and deep integration with Windows 10 features such as background tasks, notifications, tiles, and share targets.

The hosted apps feature is supported by several elements and attributes in the package manifest that enable a hosted app to use an executable and definition in a host app package. When a user runs the hosted app, the OS automatically launches the host executable under the identity of the hosted app. The host can then load visual assets, content, or call APIs as the hosted app. The hosted app gets the intersection of capabilities declared between the host and hosted app. This means that a hosted app cannot ask for more capabilities than what the host provides.

Define a host

The *host* is the main executable or runtime process for the hosted app. Currently, the only supported hosts are desktop apps (.NET or C++ desktop) that have *package identity*. There are several ways for a desktop app to have package identity:

- The most common way to grant package identity to a desktop app is by [packaging it in an MSIX package](#).
- In some cases, you may alternatively choose to grant package identity by creating a package with external location (see [Grant package identity by packaging with external location](#)). This option is useful if you're unable to adopt MSIX for installing your desktop app.

The host is declared in its package manifest by the [uap10:HostRuntime](#) extension. This extension has an **Id** attribute that must be assigned a value that is also referenced by the package manifest for the hosted app. When the hosted app is activated, the host is launched under the identity of the hosted app and can load content or binaries from the hosted app package.

The following example demonstrates how to define a host in a package manifest. The **uap10:HostRuntime** extension is package-wide and is therefore declared as a child of the **Package** element.

XML

```
<Package xmlns:uap10="http://schemas.microsoft.com/appx/manifest/uap/windows
10/10">

  <Extensions>
    <uap10:Extension Category="windows.hostRuntime"
      Executable="PyScriptEngine\PyScriptEngine.exe"
      uap10:RuntimeBehavior="packagedClassicApp"
      uap10:TrustLevel="mediumIL">
      <uap10:HostRuntime Id="PythonHost" />
    </uap10:Extension>
  </Extensions>

</Package>
```

Make note of these important details about the following elements.

 Expand table

Element	Details
uap10:Extension	The <code>windows.hostRuntime</code> category declares a package-wide extension that defines the runtime information to be used when activating a hosted app. A hosted app will run with the definitions declared in the extension. When using the host app declared in the previous example, a hosted app will run as the executable <code>PyScriptEngine.exe</code> at the <code>mediumIL</code> trust level. The <code>Executable</code>, <code>uap10:RuntimeBehavior</code>, and <code>uap10:TrustLevel</code> attributes specify the name of the host process binary in the package and how the hosted apps will run. For example, a hosted app using the attributes in the previous example will run as the executable <code>PyScriptEngine.exe</code> at <code>mediumIL</code> trust level.
uap10:HostRuntime	The <code>Id</code> attribute declares the unique identifier of this specific host app in the package. A package can have multiple host apps, and each must have a <code>uap10:HostRuntime</code> element with a unique <code>Id</code>.

Declare a hosted app

A *hosted app* declares a package dependency on a *host*. The hosted app leverages the host's ID (that is, the `Id` attribute of the **uap10:HostRuntime** extension in the host package) for activation instead of specifying an entry point executable in its own

package. The hosted app typically contains content, visual assets, scripts, or binaries that may be accessed by the host. The [TargetDeviceFamily](#) value in the hosted app package should target the same value as the host.

Hosted app packages can be signed or unsigned:

- Signed packages may contain executable files. This is useful in scenarios that have a binary extension mechanism, which enables the host to load a DLL or registered component in the hosted app package.
- In most scenarios, the unsigned package will contain executable content. But an unsigned package that contains only *non-executable* files is useful in scenarios where the host needs to load only images, assets, and content or script files. Unsigned packages must include a special `OID` value in their [Identity](#) element or they won't be allowed to register. This prevents unsigned packages from conflicting with or spoofing the identity of a signed package.

To define a hosted app, declare the following items in the package manifest:

- The [uap10:HostRuntimeDependency](#) element. This is a child of the [Dependencies](#) element.
- The [uap10:HostId](#) attribute of the [Application](#) element (for an app) or [Extension](#) element (for an activatable extension).

The following example demonstrates the relevant sections of a package manifest for an unsigned hosted app.

XML

```
<Package xmlns:uap10="http://schemas.microsoft.com/appx/manifest/uap/windows
10/10">

  <Identity Name="NumberGuesserManifest"
    Publisher="CN=AppModelSamples,
OID.2.25.311729368913984317654407730594956997722=1"
    Version="1.0.0.0" />

  <Dependencies>

    <TargetDeviceFamily Name="Windows.Desktop" MinVersion="10.0.19041.0" MaxVers
ionTested="10.0.19041.0" />

    <uap10:HostRuntimeDependency Name="PyScriptEnginePackage" Publisher="CN=AppM
odelSamples" MinVersion="1.0.0.0"/>
  </Dependencies>

  <Applications>
    <Application Id="NumberGuesserApp"
      uap10:HostId="PythonHost"
```

```

    uap10:Parameters="-Script &quot;NumberGuesser.py&quot;";>
  </Application>
</Applications>

</Package>

```

Make note of these important details about the following elements.

[Expand table](#)

Element	Details
Identity	Because the hosted app package in this example is unsigned, the Publisher attribute must include the <code>OID.2.25.311729368913984317654407730594956997722=1</code> string. This ensures that the unsigned package cannot spoof the identity of a signed package.
TargetDeviceFamily	The MinVersion attribute must specify 10.0.19041.0 or a later OS version.
uap10:HostRuntimeDependency	This element declares a dependency on the host app package. This consists of the Name and Publisher of the host package, and the MinVersion it depends on. These values can be found under the Identity element in the host package.
Application	The uap10:HostId attribute expresses the dependency on the host. The hosted app package must declare this attribute instead of the usual Executable and EntryPoint attributes for an Application or Extension element. As a result, the hosted app inherits the Executable, EntryPoint and runtime attributes from the host with the corresponding HostId value. The uap10:Parameters attribute specifies parameters that are passed to the entry point function of the host executable. Because the host needs to know what to do with these parameters, there is an implied contract between the host and hosted app.

Register an unsigned hosted app package at run time

One benefit of the **uap10:HostRuntime** extension is that it enables a host to dynamically generate a hosted app package at runtime and register it by using the [PackageManager](#) API, without needing to sign it. This enables a host to dynamically generate the content and manifest for the hosted app package and then register it.

Use the following methods of the [PackageManager](#) class to register an unsigned hosted app package. These methods are available starting in Windows 10, version 2004.

- [AddPackageByUriAsync](#): Registers an unsigned MSIX package by using the **AllowUnsigned** property of the *options* parameter.
- [RegisterPackageByUriAsync](#): Performs a loose package manifest file registration. If the package is signed, the folder containing the manifest must include a [.p7x file](#) and catalog. If unsigned, the **AllowUnsigned** property of the *options* parameter must be set.

Requirements for unsigned hosted apps

- The [Application](#) or [Extension](#) elements in the package manifest cannot contain activation data such as the **Executable**, **EntryPoint**, or **TrustLevel** attributes. Instead, these elements can only contain a **uap10:HostId** attribute that expresses the dependency on the host and a **uap10:Parameters** attribute.
- The package must be a main package. It cannot be a bundle, framework package, resource, or optional package.

Requirements for a host that installs and registers an unsigned hosted app package

- The host must have [package identity](#).
- The host must have the **packageManagement** [restricted capability](#).

XML

```
<rescap:Capability Name="packageManagement" />
```

Sample

For a fully functional sample app that declares itself as a host and then dynamically registers a hosted app package at runtime, see the [hosted app sample](#) [↗](#).

The host

The host is named **PyScriptEngine**. This is a wrapper written in C# that runs python scripts. When run with the `-Register` parameter, the script engine installs a hosted app containing a python script. When a user tries to launch the newly installed hosted app, the host is launched and executes the **NumberGuesser** python script.

The package manifest for the host app (the `Package.appxmanifest` file in the `PyScriptEnginePackage` folder) contains a **uap10:HostRuntime** extension that declares the app as a host with the ID **PythonHost** and the executable **PyScriptEngine.exe**.

ⓘ Note

In this sample, the package manifest is named `Package.appxmanifest` and it is part of a [Windows Application Packaging Project](#). When this project is built, it **generates a manifest named `AppxManifest.xml`** and builds the MSIX package for the host app.

The hosted app

The hosted app consists of a python script and package artifacts such as the package manifest. It doesn't contain any PE files.

The package manifest for the hosted app (the `NumberGuesser/AppxManifest.xml` file) contains the following items:

- The **Publisher** attribute of the **Identity** element contains the `OID.2.25.311729368913984317654407730594956997722=1` identifier, which is required for an unsigned package.
- The **uap10:HostId** attribute of the **Application** element identifies **PythonHost** as its host.

Run the sample

The sample requires version 10.0.19041.0 or later of Windows 10 and the Windows SDK.

1. Download the [sample](#) to a folder on your development computer.
2. Open the `PyScriptEngine.sln` solution in Visual Studio and set the **PyScriptEnginePackage** project as the startup project.
3. Build the **PyScriptEnginePackage** project.
4. In Solution Explorer, right-click the **PyScriptEnginePackage** project and choose **Deploy**.
5. Open a Command Prompt window to the directory where you copied the sample files and run the following command to register the sample **NumberGuesser** app

(the hosted app). Change `D:\repos\HostedApps` to the path where you copied the sample files.

CMD

```
D:\repos\HostedApps>pyscriptengine -Register  
D:\repos\HostedApps\NumberGuesser\AppxManifest.xml
```

ⓘ Note

You can run `pyscriptengine` on the command line because the host in the sample declares an [AppExecutionAlias](#).

6. Open the **Start** menu and click **NumberGuesser** to run the hosted app.

Related topics

- [Create an unsigned MSIX package](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Support your app with background tasks

Article • 10/20/2022

The topics in this section show you how to make lightweight code run in the background in response to triggers. You can use background tasks to provide functionality when your app is suspended or not running. You can also use background tasks for real-time communication apps like VOIP, mail, and IM.

Playing media in the background

Starting in Windows 10, version 1607, playing audio in the background is much easier. See [Play media in the background](#) for more information.

In-process and out-of-process background tasks

There are two approaches to implementing background tasks:

- In-process: the app and its background process run in the same process
- Out-of-process: the app and the background process run in separate processes.

In-process background support was introduced in Windows 10, version 1607, to simplify writing background tasks. But you can still write out-of-process background tasks. See [Guidelines for background tasks](#) for recommendations on when to write an in-process versus out-of-process background task.

Out-of-process background tasks are more resilient because the background process can't bring down your app process if something goes wrong. But the resiliency comes at the price of greater complexity to manage the cross-process communication between the app and the background task.

Out-of-process background tasks are implemented as lightweight classes that implement the [IBackgroundTask](#) interface that the OS runs in a separate process (backgroundtaskhost.exe). Register a background task by using the [BackgroundTaskBuilder](#) class. The class name is used to specify the entry point when you registering the background task.

In Windows 10, version 1607, you can enable background activity without creating a background task. You can instead run your background code directly inside the

foreground application's process.

To get started quickly with in-process background tasks, see [Create and register an in-process background task](#).

To get started quickly with out-of-process background tasks, see [Create and register an out-of-process background task](#).

Tip

Starting with Windows 10, you no longer need to place an app on the lock screen as a prerequisite for registering a background task for it.

Background tasks for system events

Your app can respond to system-generated events by registering a background task with the [SystemTrigger](#) class. An app can use any of the following system event triggers (defined in [SystemTriggerType](#))

Trigger name	Description
InternetAvailable	The Internet becomes available.
NetworkStateChange	A network change such as a change in cost or connectivity occurs.
OnlineIdConnectedStateChange	Online ID associated with the account changes.
SmsReceived	A new SMS message is received by an installed mobile broadband device.
TimeZoneChange	The time zone changes on the device (for example, when the system adjusts the clock for daylight saving time).

For more info see [Respond to system events with background tasks](#).

Conditions for background tasks

You can control when the background task runs, even after it is triggered, by adding a condition. Once triggered, a background task will not run until all of its conditions are met. The following conditions (represented by the [SystemConditionType](#) enumeration) can be used.

Condition name	Description
InternetAvailable	The Internet must be available.
InternetNotAvailable	The Internet must be unavailable.
SessionConnected	The session must be connected.
SessionDisconnected	The session must be disconnected.
UserNotPresent	The user must be away.
UserPresent	The user must be present.

Add the **InternetAvailable** condition to your background task

[BackgroundTaskBuilder.AddCondition](#) to delay triggering the background task until the network stack is running. This condition saves power because the background task won't execute until the network is available. This condition does not provide real-time activation.

If your background task requires network connectivity, set [IsNetworkRequested](#) to ensure that the network stays up while the background task runs. This tells the background task infrastructure to keep the network up while the task is executing, even if the device has entered Connected Standby mode. If your background task does not set **IsNetworkRequested**, then your background task will not be able to access the network when in Connected Standby mode (for example, when a phone's screen is turned off.) For more info about background task conditions, see [Set conditions for running a background task](#).

Application manifest requirements

Before your app can successfully register a background task that runs out-of-process, it must be declared in the application manifest. Background tasks that run in the same process as their host app do not need to be declared in the application manifest. For more info see [Declare background tasks in the application manifest](#).

Background tasks

The following real-time triggers can be used to run lightweight custom code in the background:

Real-time trigger	Description
-------------------	-------------

Real-time trigger	Description
Control Channel	Background tasks can keep a connection alive, and receive messages on the control channel, by using the ControlChannelTrigger . If your app is listening to a socket, you can use the Socket Broker instead of the ControlChannelTrigger . For more details on using the Socket Broker, see SocketActivityTrigger . The ControlChannelTrigger is not supported on Windows Phone.
Timer	Background tasks can run as frequently as every 15 minutes, and they can be set to run at a certain time by using the TimeTrigger . For more info see Run a background task on a timer .
Push Notification	Background tasks respond to the PushNotificationTrigger to receive raw push notifications.

Note

Universal Windows apps must call [RequestAccessAsync](#) before registering any of the background trigger types.

To ensure that your Universal Windows app continues to run properly after you release an update, call [RemoveAccess](#) and then call [RequestAccessAsync](#) when your app launches after being updated. For more information, see [Guidelines for background tasks](#).

Limits on the number of trigger instances: There are limits to how many instances of some triggers an app can register. An app can only register [ApplicationTrigger](#), [MediaProcessingTrigger](#) and [DeviceUseTrigger](#) once per instance of the app. If an app goes over this limit, registration will throw an exception.

System event triggers

The [SystemTriggerType](#) enumeration represents the following system event triggers:

Trigger name	Description
UserPresent	The background task is triggered when the user becomes present.
UserAway	The background task is triggered when the user becomes absent.
ControlChannelReset	The background task is triggered when a control channel is reset.
SessionConnected	The background task is triggered when the session is connected.

The following system event triggers signal when the user has moved an app on or off the lock screen.

Trigger name	Description
LockScreenApplicationAdded	An app tile is added to the lock screen.
LockScreenApplicationRemoved	An app tile is removed from the lock screen.

Background task resource constraints

Background tasks are lightweight. Keeping background execution to a minimum ensures the best user experience with foreground apps and battery life. This is enforced by applying resource constraints to background tasks.

Background tasks are limited to 30 seconds of wall-clock usage.

Memory constraints

Due to the resource constraints for low-memory devices, background tasks may have a memory limit that determines the maximum amount of memory the background task can use. If your background task attempts an operation that would exceed this limit, the operation will fail and may generate an out-of-memory exception--which the task can handle. If the task does not handle the out-of-memory exception, or the nature of the attempted operation is such that an out-of-memory exception was not generated, then the task will be terminated immediately.

You can use the [MemoryManager](#) APIs to query your current memory usage and limit in order to discover your cap (if any), and to monitor your background task's ongoing memory usage.

Per-device limit for apps with background tasks for low-memory devices

On memory-constrained devices, there is a limit to the number of apps that can be installed on a device and use background tasks at any given time. If this number is exceeded, the call to [RequestAccessAsync](#), which is required to register all background tasks, will fail.

Battery Saver

Unless you exempt your app so that it can still run background tasks and receive push notifications when Battery Saver is on, the Battery Saver feature, when enabled, will prevent background tasks from running when the device is not connected to external power and the battery goes below a specified amount of power remaining. This will not prevent you from registering background tasks.

However, for enterprise apps, and apps that will not be published in the Microsoft Store, see [Run in the background indefinitely](#) to learn how to use a capabilities to run a background task or extended execution session in the background indefinitely.

Background task resource guarantees for real-time communication

To prevent resource quotas from interfering with real-time communication functionality, background tasks using the [ControlChannelTrigger](#) and [PushNotificationTrigger](#) receive guaranteed CPU resource quotas for every running task. The resource quotas are as mentioned above, and remain constant for these background tasks.

Your app doesn't have to do anything differently to get the guaranteed resource quotas for [ControlChannelTrigger](#) and [PushNotificationTrigger](#) background tasks. The system always treats these as critical background tasks.

Maintenance trigger

Maintenance tasks only run when the device is plugged in to AC power. For more info see [Use a maintenance trigger](#).

Background tasks for sensors and devices

Your app can access sensors and peripheral devices from a background task with the [DeviceUseTrigger](#) class. You can use this trigger for long-running operations such as data synchronization or monitoring. Unlike tasks for system events, a **DeviceUseTrigger** task can only be triggered while your app is running in the foreground and no conditions can be set on it.

 **Important**

The `DeviceUseTrigger` and `DeviceServicingTrigger` cannot be used with in-process background tasks.

Some critical device operations, such as long running firmware updates, cannot be performed with the [DeviceUseTrigger](#). Such operations can be performed only on the PC, and only by a privileged app that uses the [DeviceServicingTrigger](#). A *privileged app* is an app that the device's manufacturer has authorized to perform those operations. Device metadata is used to specify which app, if any, has been designated as the privileged app for a device. For more info, see [Device sync and update for Microsoft Store device apps](#)

Managing background tasks

Background tasks can report progress, completion, and cancellation to your app using events and local storage. Your app can also catch exceptions thrown by a background task, and manage background task registration during app updates. For more info see:

[Handle a cancelled background task](#)

[Monitor background task progress and completion](#)

Check your background task registration during app launch. Ensure that your app's ungrouped background tasks are present in `BackgroundTaskBuilder.AllTasks`. Re-register the ones that are not present. Unregister any tasks that are no longer needed. This ensures that all background tasks registrations are up-to-date every time the app is launched.

Related topics

Conceptual guidance for multitasking in Windows 10

- [Launching, resuming, and multitasking](#)

Related background task guidance

- [Guidelines for background tasks](#)
- [Access sensors and devices from a background task](#)
- [Create and register an in-process background task](#)
- [Create and register an out-of-process background task](#)
- [Convert an out-of-process background task to an in-process background task](#)
- [Debug a background task](#)
- [Declare background tasks in the application manifest](#)

- [Group background task registration](#)
- [Handle a cancelled background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)
- [Monitor background task progress and completion](#)
- [Play media in the background](#)
- [Register a background task](#)
- [Respond to system events with background tasks](#)
- [Run a background task on a timer](#)
- [Run a background task when your UWP app is updated](#)
- [Run in the background indefinitely](#)
- [Set conditions for running a background task](#)
- [Trigger a background task from your app](#)
- [Update a live tile from a background task](#)
- [Use a maintenance trigger](#)

Guidelines for background tasks

Article • 10/20/2022

Ensure your app meets the requirements for running background tasks.

Background task guidance

Consider the following guidance when developing your background task, and before publishing your app.

If you use a background task to play media in the background, see [Play media in the background](#) for information about improvements in Windows 10, version 1607, that make it much easier.

In-process versus out-of-process background tasks: Windows 10, version 1607, introduced [in-process background tasks](#) which allows you to run background code in the same process as your foreground app. Consider the following factors when deciding whether to have in-process vs. out-of-process background tasks:

Consideration	Impact
Resilience	If your background process is running in another process, a crash in your background process won't take down your foreground application. Also, background activity can be terminated, even within your app, if it runs past execution time limits. Separating background work into a task separate from the foreground app may be a better choice when it isn't necessary for the foreground and background processes to communicate with each other (since one of the main advantages of in-process background tasks are that they remove the need for inter-process communication).
Simplicity	In-process background tasks don't require cross-process communication and are less complex to write.
Available triggers	In-process background tasks don't support the following triggers: DeviceUseTrigger , DeviceServicingTrigger and IoTStartupTask .
VoIP	In-process background tasks don't support activating a VoIP background task within your application.

Limits on the number of trigger instances: There are limits to how many instances of some triggers an app can register. An app can only register [ApplicationTrigger](#), [MediaProcessingTrigger](#) and [DeviceUseTrigger](#) once per instance of the app. If an app goes over this limit, registration will throw an exception.

CPU quotas: Background tasks are limited by the amount of wall-clock usage time they get based on trigger type. Most triggers are limited to 30 seconds of wall-clock usage, while some have the ability to run up to 10 minutes in order to complete intensive tasks. Background tasks should be lightweight to save battery life and provide a better user experience for foreground apps. See [Support your app with background tasks](#) for the resource constraints applied to background tasks.

Manage background tasks: Your app should get a list of registered background tasks, register for progress and completion handlers, and handle those events appropriately. Your background task classes should report progress, cancellation, and completion. For more info see [Handle a cancelled background task](#), and [Monitor background task progress and completion](#).

Use `BackgroundTaskDeferral`: If your background task class runs asynchronous code, make sure to use deferrals. Otherwise your background task may be terminated prematurely when the `Run` method returns (or the `OnBackgroundActivated` method in the case of in-process background tasks). For more information, see [Create and register an out-of-process background task](#).

Alternatively, request one deferral, and use `async/await` to complete asynchronous method calls. Close the deferral after the `await` method calls.

Update the app manifest: For background tasks that run out-of-process, declare each background task in the application manifest, along with the type of triggers it is used with. Otherwise your app will not be able to register the background task at runtime.

If you have multiple background tasks, consider whether they should run in the same host process or be separated into different host processes. Put them in separate host processes if you are concerned that a failure in one background task could bring down other background tasks. Use the **Resource group** entry in the manifest designer to group background tasks into different host processes.

To set the **Resource group**, open the Package.appxmanifest designer, choose **Declarations**, and add an **App Service** declaration:

Package.appxmanifest

Application Visual Assets Capabilities **Declarations** Content URIs Packaging

Use this page to add declarations and specify their properties.

Available Declarations:

App Service Add

Supported Declarations:

App Service Remove

Description:

Multiple instances of this declaration are allowed in each app.
[More information](#)

Properties:

Name:

ServerName:

☒ ExecutableOrStartPageRequired

App settings

Executable:

Entry point:

Start page:

Resource group:

See the [application schema reference](#) for more information about the resource group setting.

Background tasks that run in the same process as the foreground app do not need to declare themselves in the application manifest. For more information about declaring background tasks that run out-of-process in the manifest, see [Declare background tasks in the application manifest](#).

Prepare for app updates: If your app will be updated, create and register a **ServicingComplete** background task (see [SystemTriggerType](#)) to unregister background tasks for the previous version of the app, and register the background tasks for the new version. This is also an appropriate time to perform app updates that may be necessary outside the context of running in the foreground.

Request to execute background tasks:

Important Starting in Windows 10, apps are no longer required to be on the lock screen as a prerequisite to run background tasks.

Universal Windows Platform (UWP) apps can run all supported task types without being pinned to the lock screen. However, apps must call [GetAccessState](#) and check that the app is not denied from running in the background. Ensure that **GetAccessStatus** does not return one of the denied [BackgroundAccessStatus](#) enums. For example, this method will return **BackgroundAccessStatus.DeniedByUser** if the user has explicitly denied background task permissions for your app in the device's settings.

If your app is denied from running in the background, your app should call [RequestAccessAsync](#) and ensure the response is not denied before registering

background tasks.

For more information on user choice around background activity and Battery Saver, see [Optimize Background Activity](#).

Background task checklist

Applies to both in-process and out-of-process background tasks

- Associate your background task with the correct trigger.
- Add conditions to help ensure your background task runs successfully.
- Handle background task progress, completion, and cancellation.
- Re-register your background tasks during app launch. This ensures that they are registered the first time the app is launched. It also provides a way to detect whether the user has disabled your app's background execution capabilities (in the event registration fails).
- Check for background task registration errors. If appropriate, attempt to register the background task again with different parameter values.
- For all device families except desktop, if the device becomes low on memory, background tasks may be terminated. If an out of memory exception is not surfaced, or the app does not handle it, then the background task will be terminated without warning and without raising the `OnCanceled` event. This helps to ensure the user experience of the app in the foreground. Your background task should be designed to handle this scenario.

Applies only to out-of-process background tasks

- Create your background task in a Windows Runtime component.
- Do not display UI other than toasts, tiles, and badge updates from the background task.
- In the [Run](#) method, request deferrals for each asynchronous method call, and close them when the method is done. Or, use one deferral with **`async/await`**.
- Use persistent storage to share data between the background task and the app.
- Declare each background task in the application manifest, along with the type of triggers it is used with. Make sure the entry point and trigger types are correct.
- Do not specify an Executable element in the manifest unless you are using a trigger that should be run in the same context as the app (such as the [ControlChannelTrigger](#)).

Applies only to in-process background tasks

- When cancelling a task, ensure that the `BackgroundActivated` event handler exits before the cancellation occurs or the whole process will be terminated.
- Write background tasks that are short-lived. Most background tasks are limited to 30 seconds of wall-clock usage.

Things to avoid

- Minimize the use of inter-process communication via COM or RPC.
- The process you are trying to communicate with may not be in a running state which can result in a hang.
- A significant amount of time could be spent facilitating cross process communication, and will count against the time allotted to run your background task.
- Do not rely on user interaction in background tasks.

Related topics

- [Create and register an in-process background task](#)
- [Create and register an out-of-process background task](#)
- [Create and register a winmain COM background task](#)
- [Declare background tasks in the application manifest](#)
- [Play media in the background](#)
- [Handle a cancelled background task](#)
- [Monitor background task progress and completion](#)
- [Register a background task](#)
- [Respond to system events with background tasks](#)
- [Set conditions for running a background task](#)
- [Update a live tile from a background task](#)
- [Use a maintenance trigger](#)
- [Run a background task on a timer](#)
- [Debug a background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Access sensors and devices from a background task

Article • 10/20/2022

DeviceUseTrigger lets your Universal Windows app access sensors and peripheral devices in the background, even when your foreground app is suspended. For example, depending on where your app is running, it could use a background task to synchronize data with devices or monitor sensors. To help preserve battery life and ensure the appropriate user consent, the use of **DeviceUseTrigger** is subject to policies that are described in this topic.

To access sensors or peripheral devices in the background, create a background task that uses the **DeviceUseTrigger**. For an example that shows how this is done on a PC, see the [Custom USB device sample](#). For an example on a phone, see the [Background Sensors sample](#).

Important

DeviceUseTrigger cannot be used with in-process background tasks. The info in this topic only applies to background tasks that run out-of-process.

Device background task overview

When your app is no longer visible to the user, Windows will suspend or terminate your app to reclaim memory and CPU resources. This allows other apps to run in the foreground and reduces battery consumption. When this happens, without the help of a background task, any ongoing data events will be lost. Windows provides the background task trigger, **DeviceUseTrigger**, to let your app perform long running sync and monitoring operations on devices and sensors safely in the background, even if your app is suspended. For more info about app lifecycle, see [Launching, resuming, and background tasks](#). For more about background tasks, see [Support your app with background tasks](#).

Note In a Universal Windows app, syncing a device in the background requires that your user has approved background syncing by your app. The device must also be connected to or paired with the PC, with active I/O, and is allowed a maximum of 10 minutes of background activity. More detail on policy enforcement is described later in this topic.

Limitation: critical device operations

Some critical device operations, such as long running firmware updates, cannot be performed with the [DeviceUseTrigger](#). Such operations can be performed only on the PC, and only by a privileged app that uses the [DeviceServicingTrigger](#). A *privileged app* is an app that the device's manufacturer has authorized to perform those operations. Device metadata is used to specify which app, if any, has been designated as the privileged app for a device. For more info, see [Device sync and update for Microsoft Store device apps](#).

Protocols/APIs supported in a DeviceUseTrigger background task

Background tasks that use [DeviceUseTrigger](#) let your app communicate over many protocols/APIs, most of which aren't supported by system-triggered background tasks. The following are supported on a Universal Windows app.

Protocol	DeviceUseTrigger in a Universal Windows app
USB	✓
HID	✓
Bluetooth RFCOMM	✓
Bluetooth GATT	✓
MTP	✓
Network wired	✓
Network Wi-Fi	✓
IDeviceIOControl	✓
Sensors API	✓ (limited to sensors in the universal device family)

Registering background tasks in the app package manifest

Your app will perform sync and update operations in code that runs as part of a background task. This code is embedded in a Windows Runtime class that implements [IBackgroundTask](#) (or in a dedicated JavaScript page for JavaScript apps). To use a

DeviceUseTrigger background task, your app must declare it in the app manifest file of a foreground app, like it does for system-triggered background tasks.

In this example of an app package manifest file, **DeviceLibrary.SyncContent** is the entry point for a background task that uses the **DeviceUseTrigger**.

XML

```
<Extensions>
  <Extension Category="windows.backgroundTasks"
EntryPoint="DeviceLibrary.SyncContent">
    <BackgroundTasks>
      <m2:Task Type="deviceUse" />
    </BackgroundTasks>
  </Extension>
</Extensions>
```

Introduction to using DeviceUseTrigger

To use the **DeviceUseTrigger**, follow these basic steps. For more info about background tasks, see [Support your app with background tasks](#).

1. Your app registers its background task in the app manifest and embeds the background task code in a Windows Runtime class that implements `IBackgroundTask` or in a dedicated JavaScript page for JavaScript apps.
2. When your app starts, it will create and configure a trigger object of type **DeviceUseTrigger**, and store the trigger instance for future use.
3. Your app checks whether the background task has been previously registered and, if not, registers it against the trigger. Note that your app isn't allowed to set conditions on the task associated with this trigger.
4. When your app needs to trigger the background task, it must first call **RequestAccessAsync** to check if the app is able to request a background task.
5. If the app can request the background task, it calls the **RequestAsync** activation method on the device trigger object.
6. Your background task isn't throttled like other system background tasks (there's no CPU time quota) but will run with reduced priority to keep foreground apps responsive.
7. Windows will then validate, based on the trigger type, that the necessary policies have been met, including requesting user consent for the operation before starting the background task.
8. Windows monitors system conditions and task runtime and, if necessary, cancels the task if the required conditions are no longer met.

9. When the background tasks reports progress or completion, your app will receive these events through progress and completed events on the registered task.

Important Consider these important points when using the [DeviceUseTrigger](#):

- The ability to programmatically trigger background tasks that use the [DeviceUseTrigger](#) was first introduced in Windows 8.1 and Windows Phone 8.1.
- Certain policies are enforced by Windows to ensure user consent when updating peripheral devices on the PC.
- Additional policies are enforced to preserve user battery life when syncing and updating peripheral devices.
- Background tasks that use [DeviceUseTrigger](#) might be canceled by Windows when certain policy requirements are no longer met, including a maximum amount of background time (wall clock time). It's important to consider these policy requirements when using these background tasks to interact with your peripheral device.

Tip To see how these background tasks work, download a sample. For an example that shows how this is done on a PC, see the [Custom USB device sample](#)[↗](#). For an example on a phone, see the [Background Sensors sample](#)[↗](#).

Frequency and foreground restrictions

There is no restriction on the frequency with which your app can initiate operations, but your app can run only one [DeviceUseTrigger](#) background task operation at a time (this does not affect other types of background tasks), and can initiate a background task only while your app is in the foreground. When your app isn't in the foreground, it is unable to initiate a background task with **DeviceUseTrigger**. Your app can't initiate a second **DeviceUseTrigger** background task before the first background task has completed.

Device restrictions

While each app is limited to registering and running only one [DeviceUseTrigger](#) background task, the device (on which your app is running) may allow multiple apps to register and run **DeviceUseTrigger** background tasks. Depending on the device, there may be a limit on the total number of **DeviceUseTrigger** background tasks from all apps. This helps preserve battery on resource-constrained devices. See the following table for more details.

From a single [DeviceUseTrigger](#) background task, your app can access an unlimited number of peripheral devices or sensors - limited only by the supported APIs and protocols that were listed in the previous table.

Background task policies

Windows enforces policies when your app uses a [DeviceUseTrigger](#) background task. If these policies aren't met, the background task might be canceled. It's important to consider these policy requirements when using this type of background task to interact with devices or sensors.

Task initiation policies

This table indicates which task initiation policies apply to a Universal Windows app.

Policy	DeviceUseTrigger in a Universal Windows app
Your app is in the foreground when triggering the background task.	✓
The device is attached to the system (or in range for a wireless device).	✓
The device is accessible to the app using the supported device peripheral APIs (the Windows Runtime APIs for USB, HID, Bluetooth, Sensors, and so on). If your app can't access the device or sensor, access to the background task is denied.	✓
Background task entry point provided by the app is registered in the app package manifest.	✓
Only one DeviceUseTrigger background task is running per app.	✓
The maximum number of DeviceUseTrigger background tasks has not yet been reached on the device (on which your app is running).	Desktop device family: An unlimited number of tasks can be registered and run in parallel. Mobile device family: 1 task on a 512 MB device; otherwise, 2 tasks can be registered and run in parallel.
The maximum number of peripheral devices or sensors that your app can access from a single DeviceUseTrigger background task, when using the supported APIs/protocols.	unlimited

Policy	DeviceUseTrigger in a Universal Windows app
Your background task consumes 400ms of CPU time (assuming a 1GHz CPU) every minute when the screen is locked, or every 5 minutes when the screen is not locked. Failure to meet this policy can result in cancellation of your task.	✓

Runtime policy checks

Windows enforces the following runtime policy requirements while your task is running in the background. If any of the runtime requirements stops being true, Windows will cancel your device background task.

This table indicates which runtime policies apply to a Universal Windows app.

Policy check	DeviceUseTrigger in a Universal Windows app
The device is attached to the system (or in range for a wireless device).	✓
Task is performing regular I/O to the device (1 I/O every 5 seconds).	✓
App has not canceled the task.	✓
Wall-clock time limit – the total amount of time your app's task can run in the background.	Desktop device family: 10 minutes. Mobile device family: No time limit. To conserve resources, no more than 1 or 2 tasks can execute at once.
App has not exited.	✓

Best practices

The following are best practices for apps that use the [DeviceUseTrigger](#) background tasks.

Programming a background task

Using the [DeviceUseTrigger](#) background task from your app ensures that any sync or monitoring operations started from your foreground app continue to run in the background if your users switch apps and your foreground app is suspended by

Windows. We recommend that you follow this overall model for registering, triggering, and unregistering your background tasks:

1. Call [RequestAccessAsync](#) to check if the app is able to request a background task. This must be done before registering a background task.
2. Register the background task before requesting the trigger.
3. Connect progress and completion event handlers to your trigger. When your app returns from suspension, Windows will provide your app with any queued progress or completion events that can be used to determine the status of your background tasks.
4. Close any open device or sensor objects when you trigger your [DeviceUseTrigger](#) background task so that those devices or sensors are free to be opened and used by your background task.
5. Register the trigger.
6. Carefully consider the battery impact of accessing a device or sensor from a background task. For example, having the report interval of a sensor run too frequently could cause the task to run so often that it quickly drains a phone's battery.
7. When your background task completes, unregister it.
8. Register for cancellation events from your background task class. Registering for cancellation events will allow your background task code to cleanly stop your running background task when canceled by Windows or your foreground app.
9. On app exit (not suspension), unregister and cancel any running tasks if your app no longer needs them. On resource-constrained systems, such as low-memory phones, this will allow other apps to use a [DeviceUseTrigger](#) background task.
 - When your app exits, unregister and cancel any running tasks.
 - When your app exits, your background tasks will be canceled and any existing event handlers will be disconnected from your existing background tasks. This prevents you from determining the state of your background tasks. Unregistering and canceling the background task will allow your cancellation code to cleanly stop your background tasks.

Cancelling a background task

To cancel a task running in the background from your foreground app, use the `Unregister` method on the `BackgroundTaskRegistration` object you use in your app to register the `DeviceUseTrigger` background task. Unregistering your background task by using the `Unregister` method on `BackgroundTaskRegistration` will cause the background task infrastructure to cancel your background task.

The `Unregister` method additionally takes a Boolean true or false value to indicate if currently running instances of your background task should be canceled without allowing them to finish. For more info, see the API reference for `Unregister`.

In addition to `Unregister`, your app will also need to call `BackgroundTaskDeferral.Complete`. This informs the system that the asynchronous operation associated with a background task has finished.

Create and register an in-process background task

Article • 10/20/2022

Important APIs

- [IBackgroundTask](#)
- [BackgroundTaskBuilder](#)
- [BackgroundTaskCompletedEventHandler](#)

This topic demonstrates how to create and register a background task that runs in the same process as your app.

In-process background tasks are simpler to implement than out-of-process background tasks. However, they are less resilient. If the code running in an in-process background task crashes, it will take down your app. Also note that [DeviceUseTrigger](#), [DeviceServicingTrigger](#) and [IoTStartupTask](#) cannot be used with the in-process model. Activating a VoIP background task within your application is also not possible. These triggers and tasks are still supported using the out-of-process background task model.

Be aware that background activity can be terminated even when running inside the app's foreground process if it runs past execution time limits. For some purposes the resiliency of separating work into a background task that runs in a separate process is still useful. Keeping background work as a task separate from the foreground application may be the best option for work that does not require communication with the foreground application.

Fundamentals

The in-process model enhances the application lifecycle with improved notifications for when your app is in the foreground or in the background. Two new events are available from the Application object for these transitions: [EnteredBackground](#) and [LeavingBackground](#). These events fit into the application lifecycle based on the visibility state of your application. Read more about these events and how they affect the application lifecycle at [App lifecycle](#).

At a high level, you will handle the [EnteredBackground](#) event to run your code that will execute while your app is running in the background, and handle [LeavingBackground](#) to know when your app has moved to the foreground.

Register your background task trigger

In-process background activity is registered much the same as out-of-process background activity. All background triggers start with registration using the [BackgroundTaskBuilder](#). The builder makes it easy to register a background task by setting all required values in one place:

CS

```
var builder = new BackgroundTaskBuilder();
builder.Name = "My Background Trigger";
builder.SetTrigger(new TimeTrigger(15, true));
// Do not set builder.TaskEntryPoint for in-process background tasks
// Here we register the task and work will start based on the time trigger.
BackgroundTaskRegistration task = builder.Register();
```

ⓘ Note

Universal Windows apps must call [RequestAccessAsync](#) before registering any of the background trigger types. To ensure that your Universal Windows app continues to run properly after you release an update, you must call [RemoveAccess](#) and then call [RequestAccessAsync](#) when your app launches after being updated. For more information, see [Guidelines for background tasks](#).

For in-process background activities you do not set `TaskEntryPoint`. Leaving it blank enables the default entry point, a new protected method on the Application object called [OnBackgroundActivated\(\)](#).

Once a trigger is registered, it will fire based on the type of trigger set in the [SetTrigger](#) method. In the example above a [TimeTrigger](#) is used, which will fire fifteen minutes from the time it was registered.

Add a condition to control when your task will run (optional)

You can add a condition to control when your task will run after the trigger event occurs. For example, if you don't want the task to run until the user is present, use the condition `UserPresent`. For a list of possible conditions, see [SystemConditionType](#).

The following sample code assigns a condition requiring the user to be present:

CS

```
builder.AddCondition(new SystemCondition(SystemConditionType.UserPresent));
```

Place your background activity code in `OnBackgroundActivated()`

Put your background activity code in [OnBackgroundActivated](#) to respond to your background trigger when it fires. `OnBackgroundActivated` can be treated just like [IBackgroundTask.Run](#). The method has a [BackgroundActivatedEventArgs](#) parameter, which contains everything that the `Run` method delivers. For example, in `App.xaml.cs`:

CS

```
using Windows.ApplicationModel.Background;

...

sealed partial class App : Application
{
    ...

    protected override void OnBackgroundActivated(BackgroundActivatedEventArgs args)
    {
        base.OnBackgroundActivated(args);
        IBackgroundTaskInstance taskInstance = args.TaskInstance;
        DoYourBackgroundWork(taskInstance);
    }
}
```

For a richer `OnBackgroundActivated` example, see [Convert an app service to run in the same process as its host app](#).

Handle background task progress and completion

Task progress and completion can be monitored the same way as for multi-process background tasks (see [Monitor background task progress and completion](#)) but you will likely find that you can more easily track them by using variables to track progress or completion status in your app. This is one of the advantages of having your background activity code running in the same process as your app.